

Assignment 3: Data Warehouse Design

32558 Business Intelligence

Harsha Varthini Maniraj - 25379328

Tutorial 02

Table of Contents

Table of Contents **2**

Data Understanding and Profiling 1
Organisation: 1
Data Sources: 1
Data Profiling 1
Data Extraction 2
Data Types to Extract 2
Source Database ERD 2
Dimension Table Structures 3
Data Cleansing and Quality Assurance 4
Data Cleansing Operations 4
Raw vs Transformed Data 5
Data Loading 5
Loading Strategy 5
Data Warehouse and Analytics 6
Business Query 6
Data Warehouse Schema 6
Metadata 7
References 8

Figures

Figure 1: FilmLibrary OLTP database ERD (source system). Created using Draw.io. 2
Figure 2: FilmLibrary data warehouse Snowflake Schema. Diagram created using Miro. 6

Tables

Table 1: FilmLibrary data sources 1
Table 2: DIM_FILM - references DIM_GENRE as a snowflake sub-dimension. 3
Table 3: DIM_GENRE - snowflake branch of DIM_FILM. 3
Table 4: DIM_PLATFORM - sourced from StreamingPlatform and MovieStreamingPlatform in ERD. 3
Table 5: DIM_USER - sourced from the user accounts system. 4
Table 6: DIM_DATE - generated by the ETL pipeline. 4
Table 7: Raw source data vs transformed values. 5
Table 8: FACT_USER_ACTIVITY - stores user activity data after ETL transformation. 5
Table 9: Metadata for data elements used in the business query. 7

Data Understanding and Profiling

Organisation:

FilmLibrary is an online movie database and community platform. It contains a variety of movies where registered users can browse information, submit reviews, and review titles. The site also shows which streaming services carry each film. Data comes from studios, industry contributors, and public submissions. FilmLibrary earns revenue from Premium subscriptions, ads to free users, and affiliate commissions from streaming platforms, and aims to use data analytics to understand engagement, personalise recommendations, and identify most-liked genres and platforms.

Data Sources:

FilmLibrary uses the following internal and external data sources, as shown in Table 1.

Source Type	System	Data	Format
Internal - Structured	Movies DB	Movie, Genre, Director, Language, Country, Actor, Writer data	SQL relational DB (ERD)
Internal - Structured	User Accounts System	User profiles, subscription tier, join date	SQL relational DB
Internal - Structured	Streaming Platform Lists	StreamingPlatform and MovieStreamingPlatform data	SQL relational DB (ERD)
Internal – Semi-structured	User Submission Portal	Movie additions and movie related data additions	JSON form submissions
Internal - Semi-structured	User Activity Event Logs	Page views, ratings, review submissions	JSON log files
Internal - Unstructured	User Reviews (Review table)	Free-text review descriptions linked to MovieID	Plain text via DB
External - Unstructured	Interviews	Filmmaker statements, production notes	PDF / plain text

Table 1: FilmLibrary data sources

Data Profiling

Structure and Relationships: The FilmLibrary OLTP database stores film data in a central Movie table, connected to Genre, Director, Writer, Actor, Language, Country, and StreamingPlatform through junction tables (Figure 1). User reviews are stored in a Review table linked to Movie via MovieID. User activity events, such as ratings, are stored separately as JSON log files.

Data Quality Issues:

Missing values: Some of the movie records have a NULL Duration due to incomplete submissions. Some movies have a NULL value for the platform name as the streaming platform is unknown.

Duplicates: The user submission portal sometimes allows duplicate movie entries with slightly different titles.

Inconsistencies: Genre tags use inconsistent abbreviations. For example, 'sci-fi' vs 'Science Fiction'.

Data Extraction

Data Types to Extract

Three types of data are extracted to populate the FilmLibrary data warehouse:

- **Movies Data (Structured)** - film details including MovieName, Rating, ReleaseYear, Duration, and related entities such as Genre, Director, Language, Country, and StreamingPlatform.
  Tools and Technology: SQL queries for extraction. Integration using Apache NiFi.
- **User Activity Event Logs (Semi-structured)** - JSON records from the FilmLibrary website capturing user actions such as ratings.
  Tools and Technology: JSON parsing using Python (Pandas). Ingestion using Apache NiFi.
- **User Reviews and Press Content (Unstructured)** - free-text reviews from the Review table, plus press kits and interview transcripts submitted by studios.
  Tools and Technology: SQL queries for extraction. Processing using Natural Language Processing (NLP) via Hugging Face Transformers.

Source Database ERD

Figure 1 shows the FilmLibrary OLTP database from which structured data is extracted. This normalised structure directly informs the Snowflake Schema used in Task 5.

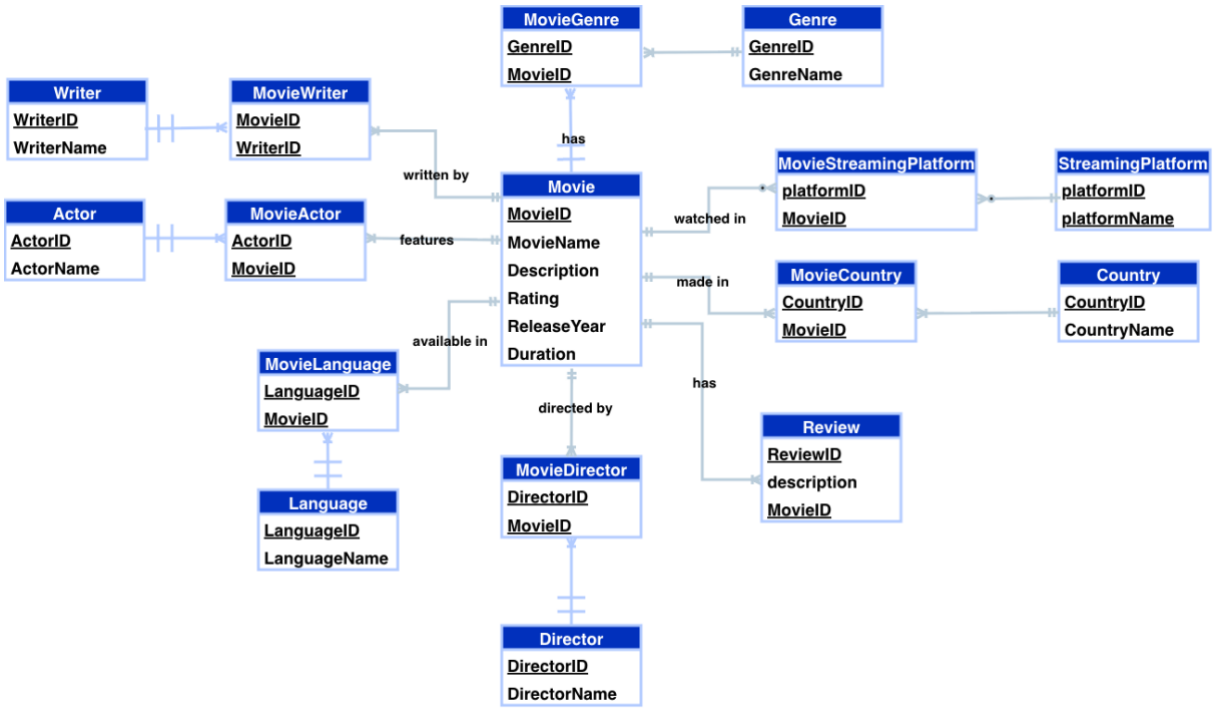


Figure 1: FilmLibrary OLTP database ERD (source system). Created using Draw.io.

Dimension Table Structures

Five dimension tables are designed based on the ERD above. DIM_FILM is the central film dimension, with DIM_GENRE as its snowflake sub-dimension.

DIM_FILM - Film Dimension (from Movie table in ERD)

Column	Data Type	Key	Description	Example
MovieID	INT	PK	Unique movie identifier	101
MovieName	VARCHAR(150)		Title of the movie	Inception
Rating	CHAR(5)		Content classification rating	PG-13
ReleaseYear	INT		Year the movie was released	2010
Duration	INT		Runtime in minutes	148
Description	TEXT		Short synopsis of the movie	A thief who steals…
GenreID	INT	FK	Links to genre sub-dimension (DIM_GENRE)	3

Table 2: DIM_FILM - references DIM_GENRE as a snowflake sub-dimension.

DIM_GENRE - Genre Dimension (from Genre and MovieGenre in ERD)

Column	Data Type	Key	Description	Example
GenreID	INT	PK	Unique genre identifier	3
GenreName	VARCHAR(50)		Full name of the genre	Science Fiction

Table 3: DIM_GENRE - snowflake branch of DIM_FILM.

DIM_PLATFORM - Streaming Platform Dimension (from MovieStreamingPlatform and StreamingPlatform tables in ERD)

Column	Data Type	Key	Description	Example
PlatformID	INT	PK	Unique platform identifier	5
PlatformName	VARCHAR(100)		Name of the streaming platform	Netflix

Table 4: DIM_PLATFORM - sourced from StreamingPlatform and MovieStreamingPlatform in ERD.

DIM_USER - User Dimension

Column	Data Type	Key	Description	Example
UserID	INT	PK	Unique user identifier	U0001
Username	VARCHAR(80)		User name chosen by the user	movielover_001
Email	VARCHAR(150)		User email address	user@email.com
JoinDate	DATE		Date the user registered for an account	2025-04-10

Table 5: DIM_USER - sourced from the user accounts system.

DIM_DATE - Date Dimension

Column	Data Type	Key	Description	Example
DateID	INT	PK	Unique Date identifier	20240615
DayName	VARCHAR(10)		Day of the week	Saturday
Month	VARCHAR(10)		Month name	June
Year	INT		Calendar year	2024

Table 6: DIM_DATE - generated by the ETL pipeline.

Data Cleansing and Quality Assurance

Data Cleansing Operations

- **Removing duplicate records:** Movie records contain duplicates as the user submissions are made more than once for the same movie; that is, slightly different titles for the same film are entered twice into the system. Python's Pandas library duplicated() Method is used to identify duplicate rows. It returns a Boolean Series indicating whether a row is a duplicate of a previous row. Then drop_duplicates() method is used to keep only one occurrence of the movie title (*Data Duplication Removal from Dataset Using Python*, 2025).
- **Standardization and Type Casting:** Genre names are inconsistent. For example, 'sci-fi' and 'Science Fiction'. This can be corrected by using a Mapping Dictionary in Python's Pandas library. All 'sci-fi', 'Sci-Fi', 'Science-Fiction' can be mapped to 'Science Fiction' (G, 2023)
- **Handling Missing Data**: Movies with NULL Duration are given a placeholder value of 0 temporarily till the correct values are added. This can be achieved using the Python method fillna(), in the Pandas library (Ngo, 2022).

Raw vs Transformed Data

Table 7 below shows how raw data is transformed.

Field	Raw Data	Transformed Data	Operation
GenreName	sci-fi (abbreviated)	Science Fiction (full term)	Python Mapping Dictionary
DuplicateRow	Duplicate movie record	Removed 1 row(1 occurrence kept)	Deduplication using duplicates()
Duration	NULL (missing)	0(placeholder for missing values)	Null handling using fillna()

Table 7: Raw source data vs transformed values.

Table 8 shows the structure of the FACT_USER_ACTIVITY fact table, which stores the transformed user activity data loaded into the data warehouse.

Column	Data Type	Key	Description	Example
ActivityID	INT	PK	Unique activity record identifier	A000001
DateID	INT	FK -> DIM_DATE	Reference to date dimension	20240615
UserID	INT	FK -> DIM_USER	Reference to user dimension	U0001
MovieID	INT	FK -> DIM_FILM	Reference to film dimension	101
PlatformID	INT	FK -> DIM_PLATFORM	Reference to platform dimension	5
UserRating	DECIMAL(3,1)		Rating given by the user (1.0 to 10.0)	8.5
ReviewSubmitted	BOOLEAN		Whether a review was submitted	TRUE

Table 8: FACT_USER_ACTIVITY - stores user activity data after ETL transformation.

Data Loading

Loading Strategy

Chosen strategy: Incremental Load.

- FilmLibrary records lots of user activities daily. A full load, replacing the entire warehouse each time, would be slow and unnecessary since historical records do not change (Chetry, 2023).
- Incremental loading only brings in new or changed records since the last load, using a LastModified timestamp on user tables.
- A regular schedule for the incremental data load process happens every night at 12:00 AM.

Data Warehouse and Analytics

Business Query

Which movies have the highest average user rating?

This helps FilmLibrary find which films users enjoy the most. The result can be used to display top-rated movies on the homepage or recommend them to new users.

The query joins FACT_USER_ACTIVITY with DIM_FILM to get the movie name, then calculates the average rating and total number of ratings per movie.

```
SELECT   df.MovieName,
         AVG(fa.UserRating)   AS AvgRating,
         COUNT(fa.ActivityID) AS TotalRatings
FROM     FACT_USER_ACTIVITY fa
JOIN     DIM_FILM df ON fa.MovieID = df.MovieID
WHERE    fa.UserRating IS NOT NULL
GROUP BY df.MovieName
ORDER BY AvgRating DESC;
```

Data Warehouse Schema

Snowflake Schema

FilmLibrary uses a Snowflake Schema. The source OLTP database (Figure 1) already stores genre in a separate table rather than inside the Movie table. The warehouse keeps this same structure, with sub-dimension tables branching off DIM_FILM.
The schema is shown in Figure 2 below. The central table is FACT_USER_ACTIVITY, which connects directly to DIM_FILM, DIM_DATE, DIM_USER, and DIM_PLATFORM via foreign keys. DIM_FILM then branches out to DIM_GENRE as a snowflake sub-dimension, shown by the dashed arrow.

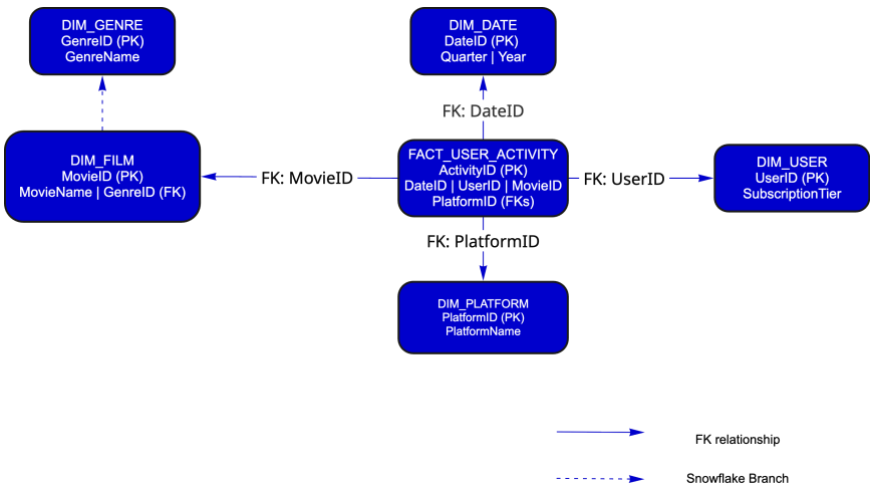


Figure 2: FilmLibrary data warehouse Snowflake Schema. Diagram created using Miro.

Query Navigation: The business query (Figure 2) starts at FACT_USER_ACTIVITY to get UserRating values, then joins to DIM_FILM to retrieve MovieName. NULL ratings are filtered out, and results are grouped by movie and ordered by average rating.

Metadata

The table below shows the metadata for the data elements used in the business query.

Metadata Element	Data Type	Source	Lineage	Access Control
UserRating	**DECIMAL(3,1)** Score from 1.0 to 10.0	FACT_USER_ACTIVITY	Activity logs -> ETL -> Fact	Analysts
MovieID/Name	**INT PK; VARCHAR(150)** Unique Movie ID and title	DIM_FILM	Movie Catalogue DB -> ETL -> DW	All authenticated users

Table 9: Metadata for data elements used in the business query.

Data Lineage: UserRating comes from the user activity event logs, is cleaned during the Transform stage, and loaded into FACT_USER_ACTIVITY via Apache NiFi. MovieName comes from the Movie table in the OLTP database and is stored in DIM_FILM.

Data Governance: UserID is personal information and is masked in query outputs for non-authorised users.

References

Chetry, N. (2023, August 12). Incremental Data Load approach and implementation strategy. *Medium*. https://medium.com/@nripapathak/incremental-data-load-approach-and-implementation-strategy-dd89bc41629c

Data Duplication Removal from Dataset Using Python. (2025). GeeksforGeeks. https://www.geeksforgeeks.org/data-analysis/data-duplication-removal-from-dataset-using-python/

G, A. (2023, August 2). *Data Cleansing & Standardization Techniques Using Python—Part 1*. Medium. https://towardsdev.com/data-cleansing-standardization-techniques-using-python-part-1-ec9134e0b497

Ngo, H. (2022, July 30). How to Clean Your Data in Python. *Towards Data Science*. https://towardsdatascience.com/how-to-clean-your-data-in-python-8f178638b98d/